



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO.



- **Alumno:**

Rodriguez Rosales Diego Guillermo.

- **Num. Control:**

#222310306.

- **Docente:**

Jesus Salas Marin.

- **Materia:**

Desarrollo de Aplicaciones para Dispositivos Móviles.

- **Clave:**

4Y8A.

- **Actividad:**

Leccion 2.4.5 Funciones Tipo.

20-Marzo-2026.





Actividad de aprendizaje.

Explorando los Tipos de Función en Dart

- Objetivo.

Declarar, almacenar y pasar funciones como parámetros utilizando tipos de función en Dart mediante ejercicios prácticos.

- Materiales:
 - ☐ Acceso a DartPad o a VisualStudio Code con Dart instalado.
 - ☐ Computadora con conexión a Internet.

Almacenar una Función en una Variable.

1. Define una función “saludar()” que imprima un mensaje de bienvenida.
2. Almacenarla en una variable “fSaludo” y llámala desde “main()”.

```
1 void saludar(String nombre) {  
2   print("¡Hola, $nombre! Bienvenido a Dart.");  
3 }  
4 void main() {  
5   // Guardamos la función en una variable  
6   void Function(String) fSaludo = saludar;  
7  
8   // Llamamos a la función a través de la variable  
9   fSaludo("Diego");  
10 }
```



Definir una Firma de Función.

1. Declara una variable "operacionMatematica" que almacene funciones que reciban dos enteros y retorna un entero.
2. Asigna una función de suma a la variable y llámala.

```
1 int sumar(int a, int b) {
2     return a + b;
3 }
4 void main() {
5     // Definir el tipo de función
6     int Function(int, int) operacionMatematica;
7     // Asignar la función 'sumar'
8     operacionMatematica = sumar;
9     print("Resultado: ${operacionMatematica(5, 3)}");
10 }
11
```

- Prueba asignando una función de resta y verificando los resultados.

```
1 int sumar(int a, int b) {
2     return a + b;
3 }
4
5 int restar(int a, int b) {
6     return a - b;
7 }
8
9 void main() {
10     // Definir el tipo de función
11     int Function(int, int) operacionMatematica;
12
13     // Asignar la función 'sumar'
14     operacionMatematica = sumar;
15     print("Suma: ${operacionMatematica(5, 3)}");
16
17     // Ahora asignar la función 'restar'
18     operacionMatematica = restar;
19     print("Resta: ${operacionMatematica(5, 3)}");
20 }
```

```
PS C:\Users\dgrr2\Downloads> dart run L2.4.5.dart
Suma: 8
Resta: 2
```



Pasar una Función como Parámetro.

1. Crea una función “ejecutarOperacion()” que reciba una función matemática y dos números.
2. Pásale una función de suma y verifica el resultado.

```
1 void ejecutarOperacion(int Function(int, int) operacion, int x, int y) {  
2   print("El resultado es: ${operacion(x, y)}");  
3 }  
4 int multiplicar(int a, int b) {  
5   return a * b;  
6 }  
7 void main() {  
8   ejecutarOperacion(multiplicar, 4, 5);  
9 }
```

- Prueba llamando la función con “sumar” y “multiplicar”.

```
1 void ejecutarOperacion(int Function(int, int) operacion, int x, int y) {  
2   print("El resultado es: ${operacion(x, y)}");  
3 }  
4  
5 int sumar(int a, int b) {  
6   return a + b;  
7 }  
8  
9 int multiplicar(int a, int b) {  
10  return a * b;  
11 }  
12  
13 void main() {  
14   // Llamando con sumar  
15   ejecutarOperacion(sumar, 4, 5);  
16  
17   // Llamando con multiplicar  
18   ejecutarOperacion(multiplicar, 4, 5);  
19 }
```

```
PS C:\Users\dgrr2\Downloads> dart run L2.4.5.dart  
El resultado es: 9  
El resultado es: 20
```



Usar Tipos de Función con Parámetros Nombrados.

1. Define una función “mostrarMensaje()” que reciba un nombre y un saludo opcional.
2. Almacenarla en una variable y llámala con y sin el parámetro greeting.

```
1 void mostrarMensaje(String nombre, {String greeting = "Hola"}) {  
2   print("$greeting, $nombre!");  
3 }  
4 void main() {  
5   void Function(String, {String greeting}) fMensaje = mostrarMensaje;  
6   fMensaje("Carlos");  
7   fMensaje("Maria", greeting: "Saludos");  
8 }
```

- Prueba cambiando los valores de greeting.

```
1 void mostrarMensaje(String nombre, {String greeting = "Hola"}) {  
2   print("$greeting, $nombre!");  
3 }  
4 void main() {  
5   void Function(String, {String greeting}) fMensaje = mostrarMensaje;  
6   fMensaje("Diego");  
7   fMensaje("Elizabeth", greeting: "Saludos");  
8 }
```

```
PS C:\Users\dgrr2\Downloads> dart run L2.4.5.dart  
Hola, Diego!  
Saludos, Elizabeth!
```



Ejercicio Final: Sistema de Notificaciones.

1. Define una función “enviarNotificacion()” que reciba un mensaje y una función de notificación.
2. Crea dos funciones: “notificarEmail()” y “notificarSMS()”.
3. Llama a “enviarNotificacion()” usando cada método.

```
1 void notificarEmail(String mensaje) {
2   print(" Enviando Email: $mensaje");
3 }
4 void notificarSMS(String mensaje) {
5   print(" Enviando SMS: $mensaje");
6 }
7 void enviarNotificacion(String mensaje, void Function(String) metodo) {
8   metodo(mensaje);
9 }
10 void main() {
11   enviarNotificacion("Tu pedido ha sido enviado.", notificarEmail);
12   enviarNotificacion("Tu código de verificación es 1234.", notificarSMS);
13 }
```

- Prueba enviando diferentes mensajes y métodos de notificación.

```
1 void notificarEmail(String mensaje) {
2   print("✉ Enviando Email: $mensaje");
3 }
4
5 void notificarSMS(String mensaje) {
6   print("📱 Enviando SMS: $mensaje");
7 }
8
9 void enviarNotificacion(String mensaje, void Function(String) metodo) {
10   metodo(mensaje);
11 }
12
13 void main() {
14   // Pruebas con Email
15   enviarNotificacion("Tu pedido ha sido enviado.", notificarEmail);
16   enviarNotificacion("Tu factura está disponible.", notificarEmail);
17
18   // Pruebas con SMS
19   enviarNotificacion("Tu código de verificación es 1234.", notificarSMS);
20   enviarNotificacion("Tu cita es mañana a las 10 AM.", notificarSMS);
21 }
```

```
PS C:\Users\dgrr2\Downloads> dart run L2.4.5.dart
✉ Enviando Email: Tu pedido ha sido enviado.
✉ Enviando Email: Tu factura está disponible.
📱 Enviando SMS: Tu código de verificación es 1234.
📱 Enviando SMS: Tu cita es mañana a las 10 AM.
```



Conclusión.

Antes de esta actividad sabía cómo crear funciones básicas en Dart, pero no entendía que también se podían almacenar en variables o pasar como parámetros. Aprendí a usar los tipos de función para hacer el código más flexible y reutilizable. Ahora entiendo cómo definir firmas de función y cómo permiten ejecutar diferentes operaciones usando la misma estructura. Me gustaría aplicar esto en proyectos más grandes para hacer programas más organizados y dinámicos.